Image Compression using Neural Networks
SETI Project 2025-2026

Louis Mennrath
Université Paris-Saclay & Télécom SudParis
louis.mennrath@telecom-sudparis.eu

Abstract

Neural image compression offers superior performance compared to traditional standards like JPEG and BPG, but remains computationally expensive for embedded devices. This report presents a lightweight decoder architecture designed for resource-constrained environments while preserving compression quality and bitrate compatibility. We explore two depthwise-separable convolution-based architectures, demonstrating that the hybrid *Mix* model achieves a 35% reduction in computational complexity with negligible quality loss compared to the state-of-the-art CompressAI model.

1 Introduction

Image compression is fundamental to modern digital communication and storage. Traditional standards such as JPEG and BPG have dominated the field for decades, but neural network-based approaches have recently demonstrated superior rate-distortion performance. However, neural compression remains computationally expensive, particularly for decompression, which typically occurs on resource-limited devices such as smartphones, IoT devices, and embedded systems where battery life and computational capacity are critical constraints.

The primary challenge is to develop a lightweight decoder that maintains compression quality and bitrate compatibility with existing standards while reducing computational overhead. This project addresses this challenge by exploring optimized architectures based on depthwise-separable convolutions.

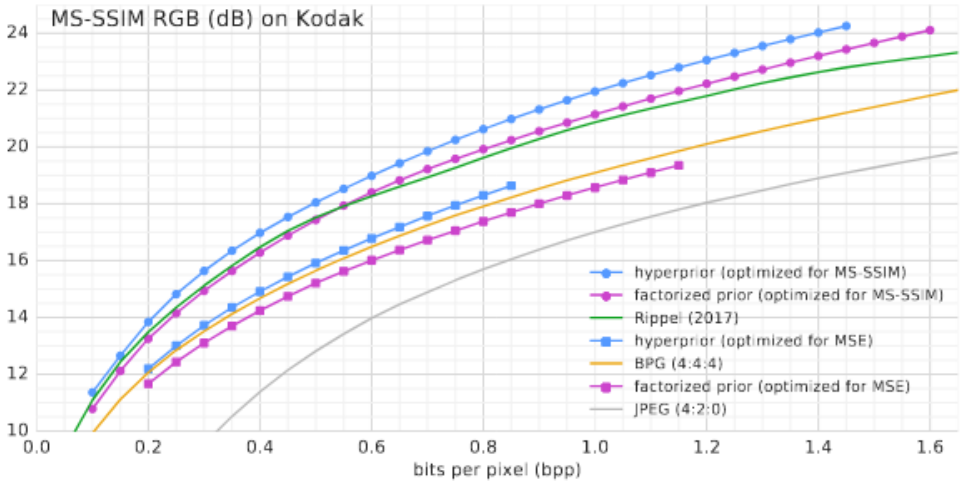

Figure 1: Comparison of image compression methods

2 Background and Methodology

2.1 Neural Image Compression Framework

We build upon the foundational work of Ballé *et al.*[1, 2], implemented through the CompressAI framework [3]. The model employs an encoder-decoder architecture:

$$\text{Encoder}: \quad L = g_a(x) \quad \text{(latent space)}$$

$$\text{Decoder}: \quad \hat{x} = g_s(L) \quad \text{(reconstruction)}$$

where g_a extracts a quantized latent representation (L) from an input image (x) and g_s reconstructs the image ($\hat{x}$). Training is governed by the loss function:

$$\mathcal{L} = R + \lambda D$$

where R represents the bitrate (bits-per-pixel) cost and D is the mean-squared error (distortion). The hyperparameter λ controls the rate-distortion trade-off. The models are trained for 8 different qualities corresponding to λ values of $[0.0018, 0.0035, 0.0067, 0.013, 0.025, 0.0483, 0.0932, 0.18]$.

2.2 Depthwise-Separable Convolutions

Standard convolutional layers process all input channels through the same set of filters, resulting in high computational complexity. Depthwise-separable convolutions decompose this operation into two steps:

1. **Depthwise**: Each input channel is convolved independently
2. **Pointwise**: A 1×1 convolution combines channels

This factorization significantly reduces the number of parameters while maintaining output dimensions.

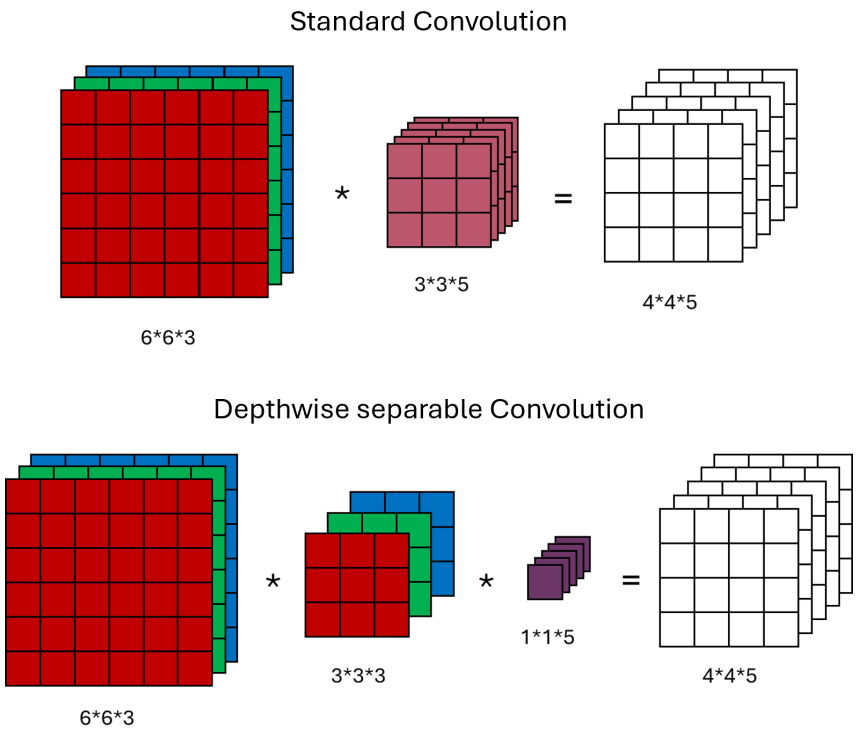

Figure 2: Depthwise-separable convolutions vs. standard convolutions

The result of the convolution is a bit different because the depthwise convolution don't take account of the channel correlation and the pointwise convolution is not able to recover all the information lost by the depthwise convolution, which can lead to a decrease in the quality of the reconstructed image. But the most important thing is that the depthwise-separable convolution can significantly reduce the number of parameters and the computational complexity of the model, which is crucial for embedded devices. We will explore the trade off between quality and computational efficiency for that application.

2.3 Proposed Approach

To see the impact of the depthwise-separable convolution we will train different decoder architectures. Rather than modifying the pre-trained encoder, we adopt a **Frozen Encoder** strategy, the encoder part of the model fixed utilizing pre-trained CompressAI weights and we modify and train only the decoder part of the model with depthwise-separable convolutions. This approach allows us to maintain bitrate compatibility and visual quality while significantly reducing the decoder's computational complexity.

3 Architecture and Experimental Setup

3.1 Model Architectures

We implemented two decoder variants based on depthwise-separable convolutions while maintaining the pre-trained encoder from COmpressAI.

3.1.1 Full Depthwise Model

All decoder convolutional layers employ depthwise-separable operations, maximizing parameter reduction.

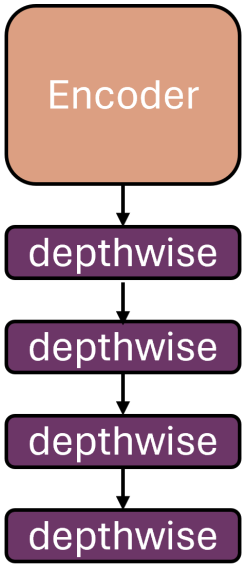

Figure 3: Full Depthwise model architecture

3.1.2 Mix Model

This hybrid approach selectively applies depthwise-separable convolutions in the firstlayers while retaining standard convolutions for the last reconstruction layers achieving a balance between efficiency and quality. The fact that the last layer is a standard convolution doesn't increase the number of parameters that much because the last reconstruction layer only have 3 channels.

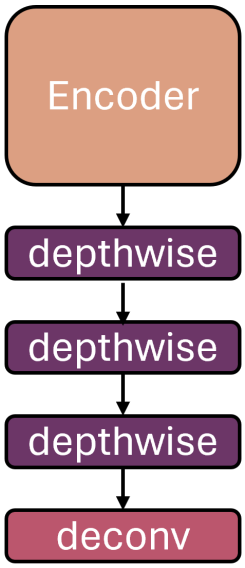

Figure 4: Mix model architecture combining standard and depthwise-separable convolutions

3.2 Training Protocol

In the compressAI documentation [5], we learn that the models are trained on the dataset Vimeo90k [6] for 4 to 5 millions steps and that is taking 2 weeks to train. For the iterations of the project we trained the models for far less steps to compare the differents architectures and see the impact of the depthwise-separable convolution on the quality of the reconstructed image. We trained the models firts on the CLIC dataset for approximately 100k steps and then we will train it on the COCO datset [7] for more than 1 million steps to precise our results.

3.3 Evaluation Protocol

The evaluation is performed on the reference Kodak dataset [4] and we mesured the performance using Peak Signal-to-Noise Ratio (PSNR) and Bits-Per-Pixel (BPP) metrics. The models are optimized for multiple λ values to cover a range of rate-distortion trade-offs, and the performance is benchmarked against the original model from Ballé *et al.* trained on the same smaller dataset with the same number of steps to ensure a fair comparison.

4 Results and Analysis

4.1 Quantitative Results

Regarding only the number of parameters we can see that the model with the full depthwise decoder is nearly 2 times lighter than the pre-trained model and the mixed model is slightly bigger the the full depthwise model but it is still 45.9% lighter than the pre-trained model.

Model	**Pretrained**	**Full Depthwise**	**Mix**
Number of Parameters	7,030,531	3,787,729	3,801,475
Parameter Reduction	0%	46.1%	45.9%

Table 1: Model size comparison

But in practice only the decoder could be implemented on the embedded device, so we will only take in consideration the number of parameters of the decoder part of the model. In that case we can see that the order of size is preserved but we achive a much more significant reduction in the number of parameters and the mixed decoder is more than **5 times lighter** than the pre-trained decoder.

Model	**Pretrained**	**Full Depthwise**	**Mix**
Number of Parameters	1,493,123	262,545	279,619
Parameter Reduction	0%	82.4%	81.3%

Table 2: Model size comparison (decoder-only)

4.2 Rate-Distortion Performance

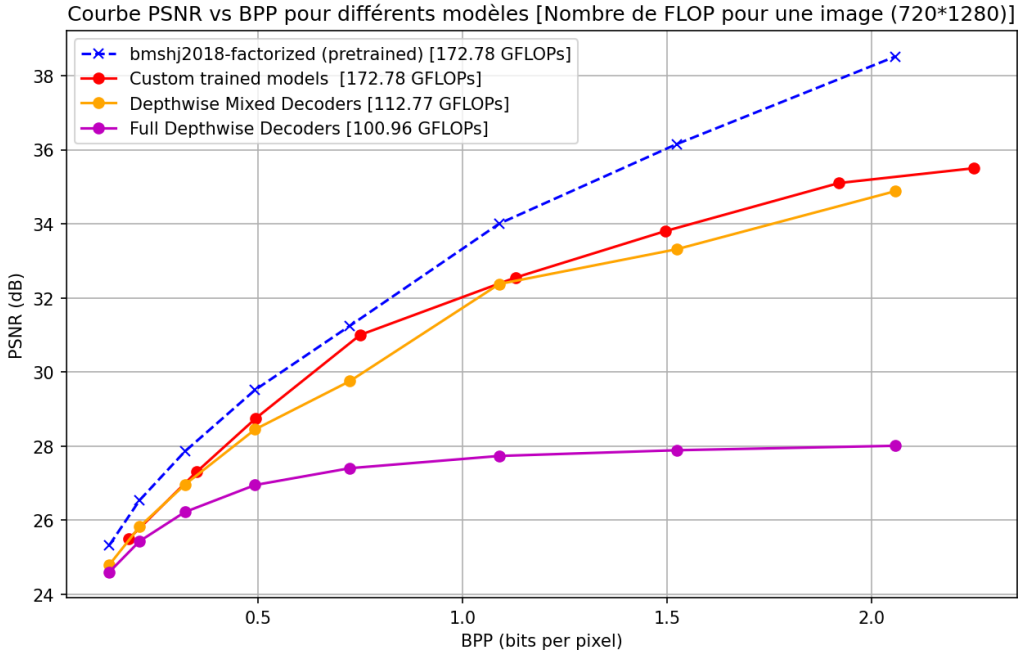

Figure 5: Rate-distortion curves across different λ values

After considering the size reduction of the model, we can test the reconstruction quality by testing the models on the Kodak dataset and plotting the rate distortion curve. The dashed blue line represnets the original pretrained model, the red line is the same model but trained on the smaller CLIC dataset and the orange and purple lines are the mixed and full depthwises models. We can see that the full depthwise model as a very low PSNR at all bitrates compared to the custom trained model but the mixed model despite being 5 times lighter than the pretrained model is able to achieve a very similar performance with a negligible loss in quality.

The flops of the model are also an important metric to take in consideration because it is a good indicator of the computational complexity of the model. We can see that on a 720*1280 image, the full depthwise model is 58% less computationally expensive than the pretrained model and the mixed model is 35% less computationally expensive than the pretrained model. If we take account only the decoder part of the model we got that FLOP improvement:

Taille d'image	**Pretrained**	**Full Depthwise**	**Mix**
256×256	2,78	0,47	0,63
512×512	11,11	1,88	2,50
768×768	25,00	4,22	5,63
1024×1024	44,44	7,50	10,00
1280×1280	69,44	11,72	15,63
1536×1536	100,00	16,88	22,50
FLOPs reduction	**0%**	**83.11%**	**77.47%**

Table 3: GFLOPs for different image sizes and percentage reduction

5 Conclusion

This project successfully demonstrates that lightweight decoder architectures using depthwise-separable convolutions can significantly reduce computational requirements for neural image compression while maintaining state-of-the-art quality and a bitrate compatibility with the standard.

5.1 Key Achievements

We observed a substantial reduction in the number of parameters (over 80% for the decoder) and computational complexity (nearly 80% FLOPs reduction) with the mixed decoder while maintaining a negligible loss in PSNR across a range of bitrates. The full depthwise model, while more efficient, suffered from a significant drop in quality, highlighting the importance of selective application of depthwise-separable convolutions.

5.2 Future Work

This work open the door for further exploration for learned image compression reduction techniques but the mixed decoder architecture needs to be trained on a much larger dataset to confirm the results. We are currently training the models on the COCO dataset for more than 1 million steps to see if the results are consistent with the ones obtained on the smaller CLIC dataset.

Ballé *et al.* published their architecture nearly 10 years ago and other architectures are available on compressAI

References

[1] J. Ballé, V. Laparra, and E. P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations (ICLR)*, 2017.

[2] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations (ICLR)*, 2018.

[3] J. Bégaint, F. Racapé, S. Feltman, and A. Pushparaja. CompressAI: A PyTorch library and evaluation platform for compression research. *arXiv preprint arXiv:2011.03029*, 2020.

[4] Eastman Kodak Company. Kodak Lossless True Color Image Suite [Dataset]. Available at: `http://r0k.us/graphics/kodak/`, 1999.

[5] CompressAI Documentation. Available at: `https://interdigitalinc.github.io/CompressAI/`, 2021.

[6] Video Enhancement with Task-Oriented Flow Tianfan Xue, Baian Chen, Jiajun Wu, Donglai Wei, William T. Freeman International Journal of Computer Vision 2019

[7] Microsoft COCO: Common Objects in Context Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, Piotr Dollár Freeman arXiv, 2014